In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import (precision_recall_curve, average_precision_score,
                             precision_score, recall_score, f1_score,
                             mean_absolute_error, mean_squared_error)

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

SEED = 42
keras.utils.set_random_seed(SEED)   # seeds python, numpy and tensorflow

df = pd.read_csv(
    r'C:\Users\jonat\OneDrive\Desktop\NU Homework\ANA500\ANA500Project\ANA500Projec
    parse_dates=['DateTime'],
    index_col='DateTime'
)

# Put every row on one regular 30-minute grid. The RNN windows below assume that
# consecutive rows are exactly 30 minutes apart. Rows already on the grid are uncha
# the few off-grid timestamps from the Marrakech join get folded into their nearest
df = df.sort_index().resample('30min').mean()

print(df.shape)
df.head()
```

```
(29631, 17)
```

Out[3]:

DateTime	zone1_Laayoune	zone2_Laayoune	zone3_Laayoune	zone4_Laayoune	zone5_La
2022-09-14 17:00:00	73.970017	121.549940	160.192634	120.981961	102
2022-09-14 17:30:00	79.182119	125.724875	161.709455	119.255935	102
2022-09-14 18:00:00	84.849262	132.626813	168.140525	121.562884	104
2022-09-14 18:30:00	92.625382	139.324748	173.786221	124.403135	107
2022-09-14 19:00:00	105.361513	160.627248	200.556432	141.767547	133

In [4]:
```
LOOKBACK = 48          # 48 half-hour readings = 24 hours of history fed to the mod
HORIZON  = 6           # forecast 6 steps ahead = 3 hours. Try 1 (30 min) for a sen
```

```
PEAK_Q    = 0.90        # peak = above the zone's 90th percentile (TRAINING period o
TRAIN_FRAC, VAL_FRAC = 0.70, 0.15     # remaining 15% is the test set
EPOCHS    = 20          # early stopping usually ends sooner; budget ~1-2 min per ep
BATCH     = 512
TRAIN_SUBSAMPLE = None # set to e.g. 50_000 for a fast dry run of the neural nets

assert LOOKBACK == 48 and HORIZON <= 48, "the 'same time yesterday' feature below a
```

In [5]:
```
amp_cols = [c for c in df.columns if 'Marrakech' not in c]
VOLTAGE, POWER_FACTOR = 220, 0.9

def amps_to_kw(current_amps, voltage=VOLTAGE, power_factor=POWER_FACTOR):
    return (voltage * current_amps * power_factor) / 1000

df_kw = df.copy()
df_kw[amp_cols] = df_kw[amp_cols].apply(amps_to_kw)

zone_cols = [c for c in df_kw.columns if c.startswith('zone')]
Z = len(zone_cols)
city_of_zone = np.array([c.split('_')[1] for c in zone_cols])
print(Z, 'zones:', zone_cols)
```

```
17 zones: ['zone1_Laayoune', 'zone2_Laayoune', 'zone3_Laayoune', 'zone4_Laayoune', '
zone5_Laayoune', 'zone1_Boujdour', 'zone2_Boujdour', 'zone3_Boujdour', 'zone1_FoumEl
oued', 'zone2_FoumEloued', 'zone3_FoumEloued', 'zone4_FoumEloued', 'zone5_FoumEloued
', 'zone6_FoumEloued', 'zone7_FoumEloued', 'zone1_Marrakech', 'zone2_Marrakech']
```

In [6]:
```
n = len(df_kw)
train_end = df_kw.index[int(n * TRAIN_FRAC)]
val_end   = df_kw.index[int(n * (TRAIN_FRAC + VAL_FRAC))]
print('train :', df_kw.index[0], '->', train_end)
print('val   :', train_end, '->', val_end)
print('test  :', val_end, '->', df_kw.index[-1])

in_train = df_kw.index <= train_end
in_val   = (df_kw.index > train_end) & (df_kw.index <= val_end)
in_test  = df_kw.index > val_end

# How many real (non-missing) readings does each zone have in each period?
coverage = pd.DataFrame({
    'train': df_kw.loc[in_train, zone_cols].notna().sum(),
    'val':   df_kw.loc[in_val,   zone_cols].notna().sum(),
    'test':  df_kw.loc[in_test,  zone_cols].notna().sum(),
})
coverage
```

```
train : 2022-09-14 17:00:00 -> 2023-11-20 19:30:00
val   : 2023-11-20 19:30:00 -> 2024-02-21 10:00:00
test  : 2024-02-21 10:00:00 -> 2024-05-24 00:00:00
```

Out[6]:

	train	val	test
zone1_Laayoune	20740	4445	4442
zone2_Laayoune	20740	4445	4442
zone3_Laayoune	20740	4445	4442
zone4_Laayoune	20740	4445	4442
zone5_Laayoune	20740	4445	4442
zone1_Boujdour	20740	4445	4442
zone2_Boujdour	20740	4445	4442
zone3_Boujdour	20740	4445	4442
zone1_FoumEloued	20740	4445	4442
zone2_FoumEloued	20740	4445	4442
zone3_FoumEloued	20740	4445	4442
zone4_FoumEloued	20740	4445	4442
zone5_FoumEloued	20740	4445	4442
zone6_FoumEloued	20740	4445	4442
zone7_FoumEloued	20740	4445	4442
zone1_Marrakech	15135	2360	0
zone2_Marrakech	15135	2360	0

In [7]:
```
thr_train = df_kw.loc[in_train, zone_cols].quantile(PEAK_Q)     # used everywhere b
thr_leaky = df_kw[zone_cols].quantile(PEAK_Q)                   # what MP3 effectiv

threshold_comparison = pd.DataFrame({'train_only': thr_train, 'full_data_MP3': thr_
threshold_comparison['shift_%'] = (threshold_comparison['train_only'] / threshold_c
display(threshold_comparison.round(2))

# Peak prevalence per period, using the TRAINING thresholds (missing readings are e
def peak_rate(mask):
    sub = df_kw.loc[mask, zone_cols]
    return ((sub > thr_train) & sub.notna()).sum() / sub.notna().sum()

prevalence = pd.DataFrame({'train': peak_rate(in_train), 'val': peak_rate(in_val),
display((prevalence * 100).round(1))
```

	train_only	full_data_MP3	shift_%
zone1_Laayoune	18.10	17.97	0.74
zone2_Laayoune	30.94	30.30	2.10
zone3_Laayoune	40.10	40.27	-0.43
zone4_Laayoune	28.58	28.49	0.30
zone5_Laayoune	31.55	32.28	-2.26
zone1_Boujdour	21.84	21.05	3.73
zone2_Boujdour	5.01	10.01	-49.90
zone3_Boujdour	13.04	12.84	1.59
zone1_FoumEloued	24.93	23.66	5.37
zone2_FoumEloued	30.42	28.48	6.83
zone3_FoumEloued	31.83	29.77	6.93
zone4_FoumEloued	15.24	14.22	7.11
zone5_FoumEloued	3.86	3.93	-1.70
zone6_FoumEloued	5.87	5.69	3.18
zone7_FoumEloued	31.05	30.14	3.01
zone1_Marrakech	1613.00	1587.00	1.64
zone2_Marrakech	352.00	356.00	-1.12

	train	val	test
zone1_Laayoune	10.0	12.6	4.1
zone2_Laayoune	10.0	7.3	2.3
zone3_Laayoune	10.0	14.6	11.1
zone4_Laayoune	10.0	7.9	8.4
zone5_Laayoune	10.0	26.5	15.9
zone1_Boujdour	10.0	2.7	0.0
zone2_Boujdour	10.0	75.8	97.1
zone3_Boujdour	10.0	5.4	8.8
zone1_FoumEloued	10.0	1.9	0.6
zone2_FoumEloued	10.0	2.7	0.5
zone3_FoumEloued	10.0	1.8	0.0
zone4_FoumEloued	10.0	2.4	0.0
zone5_FoumEloued	10.0	15.9	17.8
zone6_FoumEloued	10.0	0.6	1.5
zone7_FoumEloued	10.0	7.5	1.7
zone1_Marrakech	10.0	0.0	NaN
zone2_Marrakech	9.9	23.6	NaN

In [8]:
```
mu = df_kw.loc[in_train, zone_cols].mean()
sd = df_kw.loc[in_train, zone_cols].std()
S = ((df_kw[zone_cols] - mu) / sd).to_numpy(dtype='float32')       # (T, Z), NaN pr
thr_s = ((thr_train - mu) / sd).to_numpy(dtype='float32')          # threshold in s

idx = df_kw.index
hour = np.asarray(idx.hour + idx.minute / 60, dtype='float64')
dow  = np.asarray(idx.dayofweek, dtype='float64')
doy  = np.asarray(idx.dayofyear, dtype='float64')
T_FEATS = np.column_stack([
    np.sin(2 * np.pi * hour / 24), np.cos(2 * np.pi * hour / 24),
    np.sin(2 * np.pi * dow / 7),   np.cos(2 * np.pi * dow / 7),
    np.sin(2 * np.pi * doy / 366), np.cos(2 * np.pi * doy / 366),
]).astype('float32')

pos_split = np.where(in_train, 0, np.where(in_val, 1, 2))   # split of each row, ju

def build_samples(S):
    T = S.shape[0]
    j_all = np.arange(LOOKBACK + HORIZON - 1, T)            # target row positions
    a_all = j_all - HORIZON - LOOKBACK + 1                  # first row of the inpu
    b_all = j_all - HORIZON                                 # last row of the input
```

```
    Xs, ys, js, zs = [], [], [], []
    for z in range(S.shape[1]):
        s = S[:, z]
        ok = ~np.isnan(s)
        bad_c = np.concatenate([[0], np.cumsum(~ok)])       # bad_c[k] = number of
        keep = ((bad_c[b_all + 1] - bad_c[a_all]) == 0) & ok[j_all]
        if keep.sum() == 0:
            continue
        win = np.lib.stride_tricks.sliding_window_view(s, LOOKBACK)
        Xs.append(win[a_all[keep]]); ys.append(s[j_all[keep]])
        js.append(j_all[keep]);      zs.append(np.full(keep.sum(), z))
    return np.concatenate(Xs), np.concatenate(ys), np.concatenate(js), np.concatena

X_all, y_all, j_all, z_all = build_samples(S)
assert not np.isnan(X_all).any() and not np.isnan(y_all).any()

def make_split(code_):
    m = pos_split[j_all] == code_
    return dict(X=X_all[m], y=y_all[m], j=j_all[m], z=z_all[m],
                thr=thr_s[z_all[m]], peak=(y_all[m] > thr_s[z_all[m]]).astype(int))

train, val, test = make_split(0), make_split(1), make_split(2)
for name, d in [('train', train), ('val', val), ('test', test)]:
    print(f"{name:5s}: {len(d['y']):>7,} windows | peak rate {d['peak'].mean()*100:
```

```
train: 339,438 windows | peak rate  10.0% | cities: ['Boujdour', 'FoumEloued', 'Laay
oune', 'Marrakech']
val  :  71,395 windows | peak rate  12.3% | cities: ['Boujdour', 'FoumEloued', 'Laay
oune', 'Marrakech']
test :  65,895 windows | peak rate  11.3% | cities: ['Boujdour', 'FoumEloued', 'Laay
oune']
```

In [9]:

```
def tabular(d):
    X = d['X']
    last, mean6 = X[:, -1], X[:, -6:].mean(axis=1)
    num = np.column_stack([last, mean6, X.mean(axis=1), X.std(axis=1),
                           X[:, HORIZON - 1],                # reading 24 h before
                           last - mean6])
    zoh = np.eye(Z, dtype='float32')[d['z']]
    t = T_FEATS[d['j']]
    h = hour[d['j']]
    harm = np.column_stack([t[:, 0], t[:, 1], np.sin(4 * np.pi * h / 24), np.cos(4
    inter = (zoh[:, :, None] * harm[:, None, :]).reshape(len(h), -1)
    return np.hstack([num, zoh, t, inter]).astype('float32')

Xtab = {k: tabular(d) for k, d in [('train', train), ('val', val), ('test', test)]}
print({k: v.shape for k, v in Xtab.items()})
```

```
{'train': (339438, 97), 'val': (71395, 97), 'test': (65895, 97)}
```

In [10]:

```
scores, preds, cuts, rows = {}, {}, {}, {}

def best_f1_cut(y_true, s):
    p, r, t = precision_recall_curve(y_true, s)
    f1 = 2 * p[:-1] * r[:-1] / (p[:-1] + r[:-1] + 1e-12)
    return t[np.argmax(f1)]
```

```
def register(name, s_val, s_test, pred_test=None):
    """s_* = peak scores. pred_test = forecast in scaled units (regression models o
    scores[name] = {'val': s_val, 'test': s_test}
    cuts[name] = best_f1_cut(val['peak'], s_val)
    row = {'AP (PR-AUC)': average_precision_score(test['peak'], s_test)}
    if pred_test is not None:
        preds[name] = pred_test
        row['MAE (z)'] = mean_absolute_error(test['y'], pred_test)
        row['RMSE (z)'] = np.sqrt(mean_squared_error(test['y'], pred_test))
    for label, cut in [('default', 0.0), ('tuned', cuts[name])]:
        yhat = (s_test > cut).astype(int)
        row[f'P ({label})'] = precision_score(test['peak'], yhat, zero_division=0)
        row[f'R ({label})'] = recall_score(test['peak'], yhat, zero_division=0)
        row[f'F1 ({label})'] = f1_score(test['peak'], yhat, zero_division=0)
    rows[name] = row

# 1) Persistence: "the load in 3 hours will look like the load right now"
register('Persistence',
         val['X'][:, -1] - val['thr'], test['X'][:, -1] - test['thr'],
         pred_test=test['X'][:, -1])

# 2) Linear Regression (forecasts the scaled kW)
lin = LinearRegression().fit(Xtab['train'], train['y'])
register('Linear Regression',
         lin.predict(Xtab['val']) - val['thr'], lin.predict(Xtab['test']) - test['t
         pred_test=lin.predict(Xtab['test']))

# 3) Logistic Regression (predicts the Peak flag directly)
logreg = LogisticRegression(class_weight='balanced', max_iter=1000).fit(Xtab['train
register('Logistic Regression', logreg.decision_function(Xtab['val']), logreg.decis

# 4) Linear SVM
svm = LinearSVC(class_weight='balanced', dual=False, max_iter=2000).fit(Xtab['train
register('Linear SVM', svm.decision_function(Xtab['val']), svm.decision_function(Xt

pd.DataFrame(rows).T.round(3)
```

Out[10]:

	AP (PR-AUC)	MAE (z)	RMSE (z)	P (default)	R (default)	F1 (default)	P (tuned)	R (tuned)	F1 (tuned)
Persistence	0.771	0.549	0.811	0.676	0.677	0.677	0.744	0.659	0.699
Linear Regression	0.895	0.380	0.599	0.897	0.704	0.789	0.811	0.775	0.792
Logistic Regression	0.898	NaN	NaN	0.597	0.933	0.728	0.779	0.820	0.799
Linear SVM	0.898	NaN	NaN	0.583	0.937	0.719	0.775	0.824	0.798

In [11]:

```
def to_inputs(d, idx_=None):
    X, static = d['X'][..., None], np.hstack([np.eye(Z, dtype='float32')[d['z']], T
    y = d['y']
    if idx_ is not None:
```

```
        X, static, y = X[idx_], static[idx_], y[idx_]
    return [X, static], y

def build_rnn(cell='LSTM', units=64):
    seq_in = keras.Input(shape=(LOOKBACK, 1), name='load_sequence')
    static_in = keras.Input(shape=(Z + 6,), name='zone_and_calendar')
    Rnn = layers.LSTM if cell == 'LSTM' else layers.GRU
    x = Rnn(units)(seq_in)
    x = layers.Concatenate()([x, static_in])
    x = layers.Dense(64, activation='relu')(x)
    x = layers.Dropout(0.2)(x)
    out = layers.Dense(1)(x)
    model = keras.Model([seq_in, static_in], out, name=f'{cell}_forecaster')
    model.compile(optimizer=keras.optimizers.Adam(1e-3), loss='mse', metrics=['mae'
    return model

tr_idx = np.arange(len(train['y']))
if TRAIN_SUBSAMPLE:
    tr_idx = np.random.default_rng(SEED).choice(tr_idx, TRAIN_SUBSAMPLE, replace=Fa

Xtr, ytr = to_inputs(train, tr_idx)
Xva, yva = to_inputs(val)
Xte, _   = to_inputs(test)

histories = {}
for cell in ['LSTM', 'GRU']:
    keras.utils.set_random_seed(SEED)
    model = build_rnn(cell)
    stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=3, restore_be
    histories[cell] = model.fit(Xtr, ytr, validation_data=(Xva, yva),
                                epochs=EPOCHS, batch_size=BATCH, callbacks=[stop],
    p_val  = model.predict(Xva, batch_size=2048, verbose=0).ravel()
    p_test = model.predict(Xte, batch_size=2048, verbose=0).ravel()
    register(cell, p_val - val['thr'], p_test - test['thr'], pred_test=p_test)

results = pd.DataFrame(rows).T
results.round(3)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/20
663/663 - 40s - 60ms/step - loss: 0.2804 - mae: 0.3776 - val_loss: 0.3530 - val_mae:
0.3372
Epoch 2/20
663/663 - 36s - 55ms/step - loss: 0.1825 - mae: 0.2911 - val_loss: 0.3221 - val_mae:
0.3164
Epoch 3/20
663/663 - 36s - 55ms/step - loss: 0.1686 - mae: 0.2777 - val_loss: 0.3089 - val_mae:
0.3093
Epoch 4/20
663/663 - 37s - 55ms/step - loss: 0.1620 - mae: 0.2708 - val_loss: 0.3028 - val_mae:
0.3021
Epoch 5/20
663/663 - 37s - 55ms/step - loss: 0.1584 - mae: 0.2667 - val_loss: 0.2958 - val_mae:
0.2980
Epoch 6/20
663/663 - 36s - 54ms/step - loss: 0.1550 - mae: 0.2631 - val_loss: 0.2964 - val_mae:
0.2961
Epoch 7/20
663/663 - 36s - 55ms/step - loss: 0.1525 - mae: 0.2605 - val_loss: 0.3016 - val_mae:
0.2971
Epoch 8/20
663/663 - 38s - 57ms/step - loss: 0.1501 - mae: 0.2580 - val_loss: 0.3088 - val_mae:
0.2990
Epoch 1/20
663/663 - 39s - 58ms/step - loss: 0.2913 - mae: 0.3834 - val_loss: 0.2662 - val_mae:
0.3177
Epoch 2/20
663/663 - 36s - 55ms/step - loss: 0.1867 - mae: 0.2950 - val_loss: 0.2533 - val_mae:
0.2996
Epoch 3/20
663/663 - 35s - 53ms/step - loss: 0.1750 - mae: 0.2823 - val_loss: 0.2555 - val_mae:
0.2921
Epoch 4/20
663/663 - 35s - 53ms/step - loss: 0.1681 - mae: 0.2747 - val_loss: 0.2469 - val_mae:
0.2829
Epoch 5/20
663/663 - 35s - 53ms/step - loss: 0.1643 - mae: 0.2702 - val_loss: 0.2566 - val_mae:
0.2835
Epoch 6/20
663/663 - 35s - 53ms/step - loss: 0.1609 - mae: 0.2667 - val_loss: 0.2504 - val_mae:
0.2808
Epoch 7/20
663/663 - 35s - 53ms/step - loss: 0.1591 - mae: 0.2651 - val_loss: 0.2537 - val_mae:
0.2803
```

Out[11]:

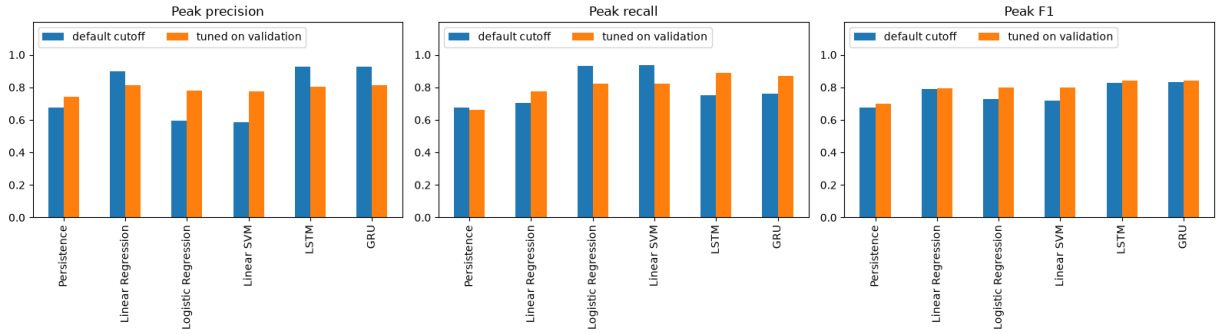
	AP (PR-AUC)	MAE (z)	RMSE (z)	P (default)	R (default)	F1 (default)	P (tuned)	R (tuned)	F1 (tuned)
Persistence	0.771	0.549	0.811	0.676	0.677	0.677	0.744	0.659	0.699
Linear Regression	0.895	0.380	0.599	0.897	0.704	0.789	0.811	0.775	0.792
Logistic Regression	0.898	NaN	NaN	0.597	0.933	0.728	0.779	0.820	0.799
Linear SVM	0.898	NaN	NaN	0.583	0.937	0.719	0.775	0.824	0.798
LSTM	0.933	0.396	0.781	0.925	0.752	0.829	0.804	0.887	0.843
GRU	0.930	0.363	0.704	0.924	0.759	0.833	0.815	0.868	0.841

In [19]:
```
# Model comparison on the shared test period
print(f"Test windows: {len(test['y']):,} | test Peak prevalence: {test['peak'].mean
      f"(this is the PR-AUC value a model with no skill would get)")
display(results.round(3))

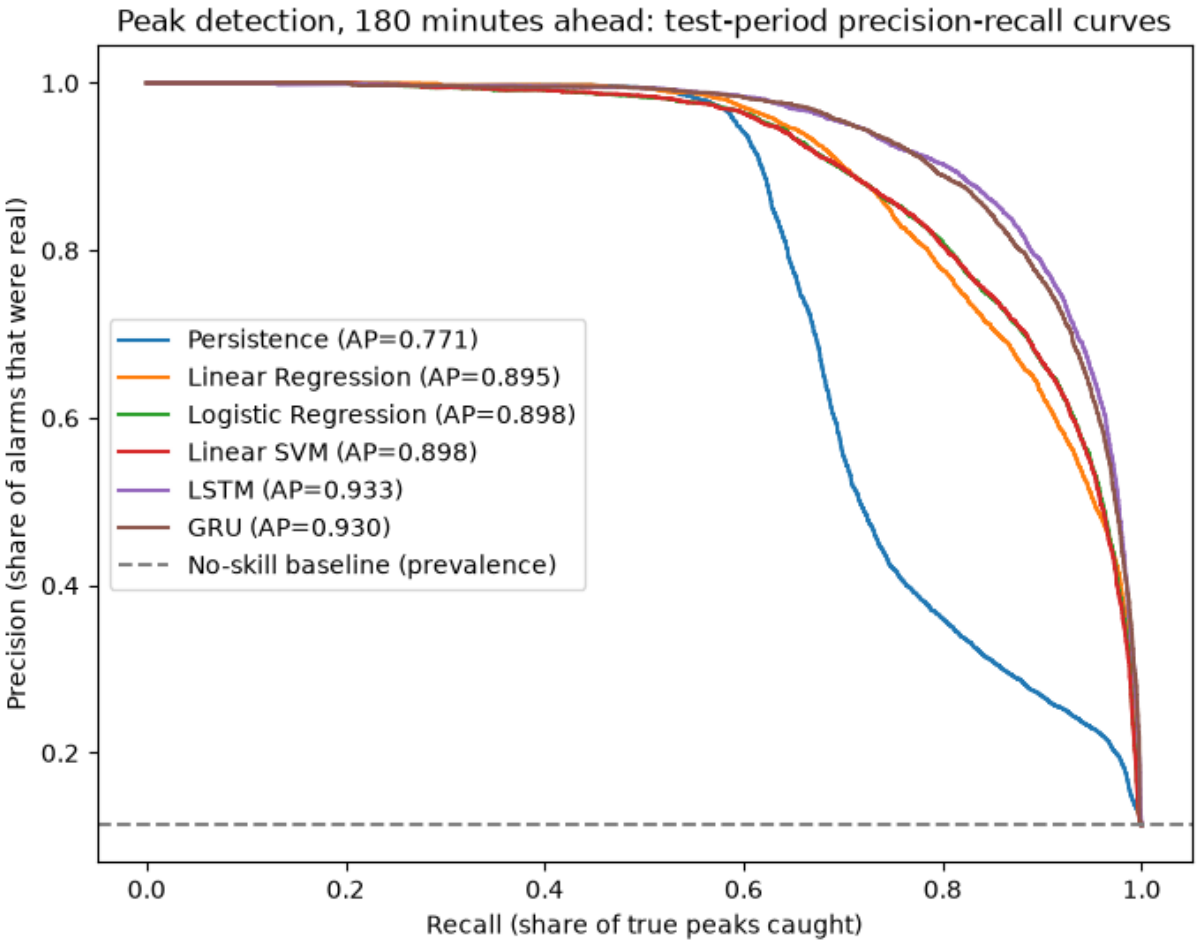
# How much of the nominal peak-detection ability survives at the default 'score > 0
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
for ax, metric in zip(axes, ['P', 'R', 'F1']):
    plot_df = results[[f'{metric} (default)', f'{metric} (tuned)']]
    plot_df.columns = ['default cutoff', 'tuned on validation']
    plot_df.plot(kind='bar', ax=ax)
    ax.set_title({'P': 'Peak precision', 'R': 'Peak recall', 'F1': 'Peak F1'}[metri
    ax.set_ylim(0, 1.2); ax.set_yticks(np.arange(0, 1.01, 0.2)); ax.legend(loc='upp
plt.tight_layout(); plt.show()
```

```
Test windows: 65,895 | test Peak prevalence: 11.3% (this is the PR-AUC value a model
with no skill would get)
```


	AP (PR-AUC)	MAE (z)	RMSE (z)	P (default)	R (default)	F1 (default)	P (tuned)	R (tuned)	F1 (tuned)
Persistence	0.771	0.549	0.811	0.676	0.677	0.677	0.744	0.659	0.699
Linear Regression	0.895	0.380	0.599	0.897	0.704	0.789	0.811	0.775	0.792
Logistic Regression	0.898	NaN	NaN	0.597	0.933	0.728	0.779	0.820	0.799
Linear SVM	0.898	NaN	NaN	0.583	0.937	0.719	0.775	0.824	0.798
LSTM	0.933	0.396	0.781	0.925	0.752	0.829	0.804	0.887	0.843
GRU	0.930	0.363	0.704	0.924	0.759	0.833	0.815	0.868	0.841

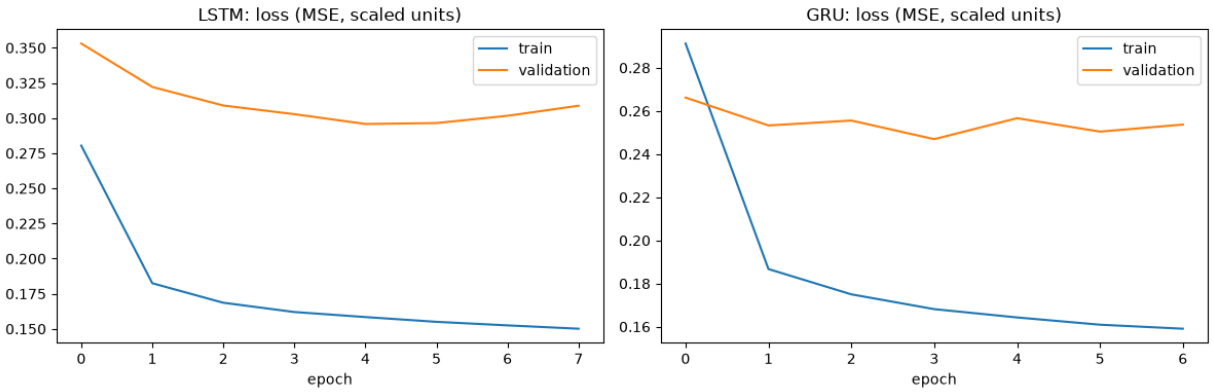


In [13]:
```
# Precision-recall curves on the test set (no cutoff epoch chosen: shows full trade
plt.figure(figsize=(8, 6))
for name in scores:
    p, r, _ = precision_recall_curve(test['peak'], scores[name]['test'])
    plt.plot(r, p, label=f"{name} (AP={results.loc[name, 'AP (PR-AUC)']:.3f})")
plt.axhline(test['peak'].mean(), color='gray', ls='--', label='No-skill baseline (p
plt.xlabel('Recall (share of true peaks caught)'); plt.ylabel('Precision (share of
plt.title(f'Peak detection, {HORIZON * 30} minutes ahead: test-period precision-rec
plt.legend(); plt.show()
```



In [14]:
```
# Train and Validation loss curves for the RNNs
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, cell in zip(axes, histories):
    h = histories[cell].history
    ax.plot(h['loss'], label='train'); ax.plot(h['val_loss'], label='validation')
    ax.set_title(f'{cell}: loss (MSE, scaled units)'); ax.set_xlabel('epoch'); ax.l
```

```
plt.tight_layout(); plt.show()
```



In [15]:
```
# Per city breakdown since each one behaves differently (different usage/draw at di
test_city = city_of_zone[test['z']]
city_rows = []
for name in scores:
    yhat = (scores[name]['test'] > cuts[name]).astype(int)
    for city in sorted(set(test_city)):
        m = test_city == city
        city_rows.append({'model': name, 'city': city, 'n_peaks': int(test['peak'][
                          'precision': precision_score(test['peak'][m], yhat[m], ze
                          'recall': recall_score(test['peak'][m], yhat[m], zero_div
                          'AP': average_precision_score(test['peak'][m], scores[nam
city_df = pd.DataFrame(city_rows)
for metric in ['precision', 'recall', 'AP']:
    print(f'--- {metric} ---')
    display(city_df.pivot(index='model', columns='city', values=metric).round(3))
```

```
--- precision ---
```

model	Boujdour	FoumEloued	Laayoune
GRU	0.967	0.558	0.609
LSTM	0.965	0.599	0.580
Linear Regression	0.950	0.456	0.578
Linear SVM	0.938	0.498	0.524
Logistic Regression	0.935	0.503	0.533
Persistence	0.959	0.188	0.288

```
--- recall ---
```

city	Boujdour	FoumEloued	Laayoune
model			
GRU	0.981	0.596	0.725
LSTM	0.982	0.660	0.767
Linear Regression	0.980	0.483	0.409
Linear SVM	0.984	0.610	0.532
Logistic Regression	0.984	0.601	0.522
Persistence	0.949	0.138	0.203

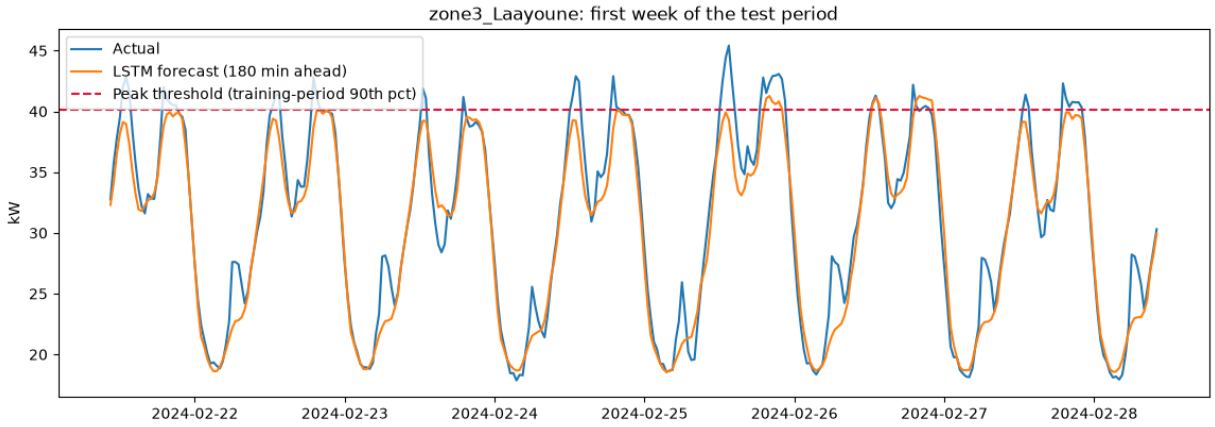
```
--- AP ---
```

city	Boujdour	FoumEloued	Laayoune
model			
GRU	0.995	0.577	0.682
LSTM	0.995	0.633	0.682
Linear Regression	0.993	0.458	0.531
Linear SVM	0.990	0.489	0.531
Logistic Regression	0.990	0.494	0.531
Persistence	0.992	0.108	0.229

In [16]:
```
# what a generated forecast will look like with this model
best_nn = max(['LSTM', 'GRU'], key=lambda m: results.loc[m, 'AP (PR-AUC)'])
zone_to_plot = 'zone3_Laayoune'            # change to any zone that has test data
zi = zone_cols.index(zone_to_plot)

m = test['z'] == zi
order = np.argsort(test['j'][m])
jj = test['j'][m][order]
week = jj < jj[0] + 7 * 48
actual_kw = test['y'][m][order] * sd.iloc[zi] + mu.iloc[zi]
pred_kw   = preds[best_nn][m][order] * sd.iloc[zi] + mu.iloc[zi]
times = df_kw.index[jj]

plt.figure(figsize=(14, 4.5))
plt.plot(times[week], actual_kw[week], label='Actual')
plt.plot(times[week], pred_kw[week], label=f'{best_nn} forecast ({HORIZON * 30} min
plt.axhline(thr_train[zone_to_plot], color='crimson', ls='--', label='Peak threshol
plt.title(f'{zone_to_plot}: first week of the test period'); plt.ylabel('kW'); plt.
```



In [17]:
```
# re-scoring without zone2_Boujdour because it caused a massive shift in the data (
zi = zone_cols.index('zone2_Boujdour')
is_b2 = test['z'] == zi
print(f"zone2_Boujdour = {is_b2.mean()*100:.1f}% of test windows and {test['peak'][

keep = ~is_b2
rows_ex = {}
for name in scores:
    s = scores[name]['test'][keep]
    yk = test['peak'][keep]
    yhat = (s > cuts[name]).astype(int)
    rows_ex[name] = {'AP': average_precision_score(yk, s),
                     'P': precision_score(yk, yhat, zero_division=0),
                     'R': recall_score(yk, yhat, zero_division=0),
                     'F1': f1_score(yk, yhat, zero_division=0)}
print(f"Test prevalence without zone2_Boujdour: {test['peak'][keep].mean()*100:.1f}
pd.DataFrame(rows_ex).T.round(3)
```

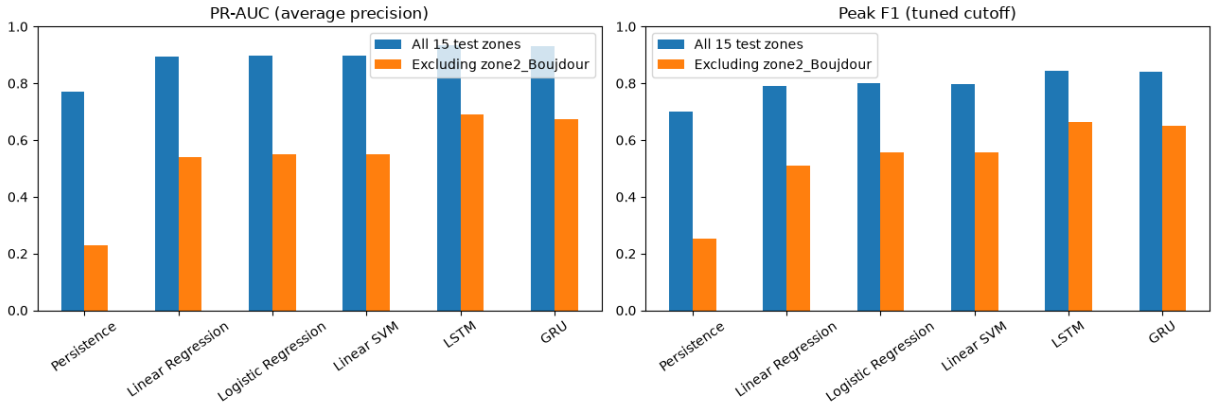
```
zone2_Boujdour = 6.7% of test windows and 57.1% of all test peaks
Test prevalence without zone2_Boujdour: 5.2%
```

Out[17]:

	AP	P	R	F1
Persistence	0.228	0.297	0.218	0.252
Linear Regression	0.541	0.548	0.478	0.510
Logistic Regression	0.551	0.531	0.584	0.556
Linear SVM	0.550	0.527	0.591	0.557
LSTM	0.689	0.601	0.739	0.663
GRU	0.675	0.609	0.694	0.649

In [18]:
```
# plot both versions side by side (excluding zone2_Boujdour and all zones and citie
ex = pd.DataFrame(rows_ex).T
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
pairs = [('AP', 'AP (PR-AUC)', 'PR-AUC (average precision)'), ('F1', 'F1 (tuned)',
for ax, (ex_col, all_col, title) in zip(axes, pairs):
    pd.DataFrame({'All 15 test zones': results[all_col].astype(float),
                  'Excluding zone2_Boujdour': ex[ex_col].astype(float)}).plot(kind=
    ax.set_title(title); ax.set_ylim(0, 1); ax.set_xlabel(''); ax.tick_params(axis=
```

```
plt.tight_layout(); plt.show()
```



In [ ]: